MINI PROJECT REPORT

SarthiGo
Car Rental Web App

Search, book and manage car rentals with Django, Tailwind CSS and SQLite

Subject: Python and Web Programming (202045612)

Submitted by
Mayuri Dudhat
Enrollment No. 12402080601078

ADIT

SarthiGo - Your Journey, Our Chariot

Certificate

This is to certify that Mayuri Dudhat, Enrollment No. 12402080601078, of ADIT has successfully completed the mini project titled SarthiGo - Car Rental Web App for the subject Python and Web Programming (202045612). The project was developed using Python, the Django framework, Tailwind CSS and an SQLite database, and it is submitted in partial fulfilment of the course requirements.

Project Guide: _____

Head of Department: _____

Date: _____

Acknowledgement

I would like to thank my subject teacher and project guide for the direction, feedback and encouragement given throughout this mini project. Their suggestions helped me turn a simple idea into a working web application.

I am also grateful to my institute, ADIT, for providing the laboratory facilities and the learning environment needed to complete the work. I thank the Django and Tailwind CSS communities, whose documentation made it much easier to learn these tools. Finally, I thank my family and friends for their constant support and for testing the application and giving honest opinions.

Mayuri Dudhat

Abstract

SarthiGo is a web application for renting cars in Anand. Traditional car rental is often handled by phone calls and paper registers, which makes it hard for a customer to know which vehicles are free on a given date and hard for the owner to track bookings. SarthiGo solves this by letting a customer enter a start date, an end date and the number of seats needed, and then showing only the cars that are actually free for that period.

A registered customer can log in, choose a car, enter driver details, accept the terms and receive an instant confirmation. The customer can later open the My Bookings page to see the full booking history. Staff members get a separate Admin Dashboard that lists the bookings of all users and allows a booking to be cancelled after a confirmation pop-up.

The system is built with Python and the Django framework using the Model-View-Template pattern. The pages are styled with Tailwind CSS loaded from a CDN, a little JavaScript checks dates in the browser, and all data is stored in SQLite, Django's default database. The total cost is calculated as the price per day multiplied by the number of days, where both the start day and the end day are counted. A cancelled booking is not deleted; its approval flag is set to false so that history is preserved.

This report describes the problem, the technology, the analysis and design, the implementation, the testing and the possible future improvements of the project.

Table of Contents

Certificate 2

Acknowledgement 3

Abstract 3

Table of Contents 4

1. Introduction 6

1.1 Background . . . 6

1.2 Problem Statement . . . 6

1.3 Objectives . . . 6

1.4 Scope . . . 6

1.5 Organization of the Report . . . 7

2. Technology Stack 7

2.1 Python . . . 7

2.2 Django . . . 7

2.3 HTML, Tailwind CSS and JavaScript . . . 7

2.4 SQLite . . . 8

2.5 Development Tools . . . 8

3. System Analysis 8

3.1 Existing System . . . 8

3.2 Proposed System . . . 8

3.3 Users and Roles . . . 8

3.4 Functional Requirements . . . 9

3.5 Non-Functional Requirements . . . 9

3.6 Hardware and Software Requirements . . . 9

3.7 Feasibility Study . . . 9

4. System Design 10

4.1 Architecture: Model-View-Template . . . 10

4.2 Project Structure . . . 10

4.3 Database Design . . . 10

4.4 Relationships . . . 11

4.5 URL Design . . . 11
4.6 Booking Workflow . . . 11
4.7 Security Design . . . 12

5. Implementation **12**

5.1 Project Settings . . . 12
5.2 Models . . . 13
5.3 Checking Availability . . . 13
5.4 Calculating the Cost . . . 14
5.5 Forms and Validation . . . 14
5.6 Views . . . 14
5.7 Admin Dashboard and Cancellation . . . 15
5.8 Templates . . . 16
5.9 Browser-Side Date Check . . . 16
5.10 Sample Data Command . . . 17

6. User Interface Description **17**

6.1 Home Page . . . 17
6.2 Available Cars Page . . . 17
6.3 Login and Sign-Up Pages . . . 17
6.4 Confirm Booking Page . . . 17
6.5 Booking Confirmation Page . . . 18
6.6 My Bookings Page . . . 18
6.7 Admin Dashboard . . . 18

7. Testing **18**

7.1 Testing Strategy . . . 18
7.2 Test Cases . . . 18
7.3 Result of Testing . . . 19

8. Installation and Execution **19**

9. Limitations **19**

10. Future Scope **19**

11. Conclusion **20**

12. References **20**

1. Introduction

1.1 Background

Car rental has become a common choice for people who need a vehicle for a weekend trip, a wedding, a family visit or a short business journey. In smaller cities such as Anand, many rental businesses still depend on phone calls, messages and handwritten registers. A customer has to call several shops to learn which car is free, and the owner has to remember or look up which cars are already promised for which dates.

Web applications can remove most of this effort. A well designed site can answer the single most important question, which is whether a car is available between two dates, within a few seconds and without human help. It can also keep an accurate record of every booking and let the owner see everything in one table.

1.2 Problem Statement

The problem addressed by this project can be stated as follows: design and build a web application in which customers can search for cars that are free on their chosen dates, book one with the required driver details, and view their past bookings, while staff can monitor and cancel bookings from a dedicated dashboard.

The main difficulties that the application must handle are listed below.

- A car must never be shown as available if it already has an approved booking that overlaps the requested dates.
- Dates entered by the user must be valid: the start date cannot be in the past and the end date cannot be earlier than the start date.
- Only logged-in users may book, and only staff may see all bookings or cancel them.
- The total cost must be calculated in a consistent and clearly explained way.
- Cancelling a booking should not erase the record, because the history is useful for both the customer and the owner.

1.3 Objectives

The objectives of the project are to:

1. Provide a simple search form that accepts a start date, an end date and the number of seats.
2. Show only the cars that have enough seats and are free for the whole period.
3. Allow users to register, log in and log out securely.
4. Collect driver name, date of birth and licence number, and require acceptance of the terms before a booking is confirmed.
5. Calculate the total cost automatically and show it before confirmation.
6. Give every user a My Bookings page with the full history and status of bookings.
7. Give staff an Admin Dashboard to view all bookings and cancel them with a confirm pop-up.
8. Practise the Model-View-Template pattern, form validation, authentication and database design using Python and Django.

1.4 Scope

The scope of SarthiGo covers the complete journey from searching to booking and managing bookings for a single rental business located in Anand. It includes user accounts, a car catalogue, availability search, booking, booking history and a staff dashboard.

Online payment, driver verification through external services, car delivery tracking, e-mail or SMS notifications and mobile applications are outside the scope of this mini project. They are discussed as future

enhancements in Chapter 10.

1.5 Organization of the Report

Chapter 2 introduces the technologies used. Chapter 3 analyses the existing and proposed systems and lists the requirements. Chapter 4 presents the architecture, the database design and the URL design. Chapter 5 explains the implementation with code. Chapter 6 describes the user interface. Chapter 7 documents the testing. Chapter 8 explains how to install and run the project. Chapters 9 and 10 discuss limitations and future scope, and Chapter 11 concludes the report.

2. Technology Stack

SarthiGo uses a small and widely taught set of technologies. Each one was chosen because it is free, well documented and suitable for a student project that can still be extended later.

Layer	Technology	Purpose in SarthiGo
Backend	Python and Django	URLs, views, forms, login and booking logic
Frontend	HTML templates, Tailwind CSS (CDN), JavaScript	Page layout, styling and browser-side date checks
Database	SQLite	Stores users, cars and bookings in one file

2.1 Python

Python is a high-level, general-purpose language known for readable syntax. It has a very large standard library and a huge ecosystem of packages. In this project Python is used for every piece of server-side logic: calculating the number of days, finding free cars, validating forms and creating booking records. Because Python code reads almost like plain English, the booking rules in the views and models are easy to explain and to review.

2.2 Django

Django is a full-featured Python web framework that follows the principle of batteries included. It provides an object-relational mapper (ORM), a URL router, a template engine, form handling, a user authentication system, an automatic administration site, protection against common attacks such as cross-site request forgery (CSRF) and SQL injection, and a development server.

The features of Django that are used most in this project are:

- **ORM and migrations:** Python classes describe the tables, and migration files create them in the database.
- **Authentication:** the built-in User model, login and logout views, password hashing and the login-required decorator.
- **Forms:** form classes validate the search dates, the sign-up data and the driver details on the server.
- **Templates:** HTML pages that extend one base template and receive data from the views.
- **Management commands:** a custom seed_cars command adds sample cars to the database.
- **Admin site:** the Car and Booking models are also registered in Django's own admin interface at the admin path.

2.3 HTML, Tailwind CSS and JavaScript

The user interface is made of HTML templates. Tailwind CSS is a utility-first styling framework in which the design is built by adding small classes such as padding, colour and rounded-corner classes directly to the elements. It is loaded from a CDN, so no build step or local installation is required, which keeps the project easy to run in a laboratory.

JavaScript is used for one focused purpose: checking the dates on the home page before the form is sent. If the start date is in the past or the end date is before the start date, an alert is shown and the form is not submitted. The same checks are repeated on the server, because browser-side checks can always be bypassed.

2.4 SQLite

SQLite is a lightweight relational database that stores everything in a single file, here named db.sqlite3. It needs no separate server and no configuration, and it is the default database of Django. For a project of this size, with a few thousand records at most, SQLite is more than sufficient. Because Django's ORM hides the database details, the project can be moved to PostgreSQL or MySQL later by changing only the settings.

2.5 Development Tools

The project was developed in a code editor on a computer with Python 3 installed, using pip to install Django. The built-in Django development server was used for running and demonstrating the site, and a web browser was used for manual testing. No other external services are needed.

3. System Analysis

3.1 Existing System

In the existing manual system, a customer contacts the rental shop by phone or in person, explains the dates and the number of passengers, and waits for the owner to check a register or memory. The shop then quotes a price orally. Driver details are written on paper, and cancellations are handled through another phone call.

The drawbacks of this approach are clear.

- Availability is checked by a person, so it is slow and can be wrong.
- The customer cannot compare cars without calling several times.
- Double bookings are possible when two customers ask for the same car at nearly the same time.
- Records are scattered across paper and messages, so the owner cannot easily see all bookings.
- Price calculation is manual and can differ from one quote to another.

3.2 Proposed System

The proposed system, SarthiGo, moves the entire process online. The customer enters dates and seats and instantly sees only the free cars together with the total price for the chosen period. After logging in, the customer confirms the booking in a few clicks and can see it later in My Bookings. The owner opens the Admin Dashboard to view every booking in one table and cancels a booking with one button.

The advantages of the proposed system are speed, accuracy, a single source of truth for all records, automatic price calculation, and protection against double booking because the availability check is repeated just before the booking is saved.

3.3 Users and Roles

Role	Description	Main actions
Visitor	A person who has not logged in	View the home page, search for cars, register, log in
Customer	A logged-in user	Book a car, view My Bookings, log out
Staff / Admin	A user with the staff flag	Open the Admin Dashboard, view all bookings, cancel bookings

3.4 Functional Requirements

ID	Requirement
FR1	The system shall let a visitor search by start date, end date and number of seats.
FR2	The system shall list only cars with enough seats that have no approved booking overlapping the chosen dates.
FR3	The system shall let a new user register with a username, e-mail and password.
FR4	The system shall require login before a booking can be made.
FR5	The system shall collect driver name, date of birth and licence number and require acceptance of the terms.
FR6	The system shall calculate total cost as price per day multiplied by the number of days, counting both the start and end day.
FR7	The system shall show a confirmation page after a successful booking.
FR8	The system shall let a customer view all of his or her bookings with their status.
FR9	The system shall let staff view the bookings of all users.
FR10	The system shall let staff cancel a booking after a confirmation pop-up, keeping the record with a Canceled status.

3.5 Non-Functional Requirements

- **Usability:** the pages are simple, responsive and usable on both desktop and mobile screens.
- **Security:** passwords are hashed by Django, forms are protected by CSRF tokens, and staff pages are restricted to staff accounts.
- **Reliability:** dates are validated on both client and server, and availability is checked again at booking time.
- **Maintainability:** the code follows the standard Django project layout, with models, views, forms, URLs and templates kept in separate files.
- **Portability:** the project runs on any system that has Python, and no database server has to be installed.
- **Performance:** a search is a single database query, which is fast for the data size expected.

3.6 Hardware and Software Requirements

Item	Requirement
Processor	Any modern dual-core processor
Memory	4 GB RAM or more
Storage	200 MB of free disk space
Operating system	Windows, Linux or macOS
Language	Python 3.10 or later
Framework	Django (installed through requirements.txt)
Browser	Any current browser such as Chrome, Edge or Firefox
Internet	Needed to load Tailwind CSS from the CDN

3.7 Feasibility Study

Technical feasibility. All the tools are free and are well supported on common computers. The required skills, which are Python, HTML and basic SQL concepts, are covered in the course.

Economic feasibility. The project needs no paid software, licences or servers, so the development cost is practically zero.

Operational feasibility. The interface follows familiar patterns such as a search form, a list of cards and a table. Both customers and staff can use it without special training.

4. System Design

4.1 Architecture: Model-View-Template

Django organises a web application using the Model-View-Template (MVT) pattern, which is a variation of MVC.

- **Model:** Python classes that describe the data and are mapped to database tables. SarthiGo has two models, Car and Booking, besides Django's built-in User.
- **View:** Python functions that receive a request, apply the business rules, talk to the models and return a response. Examples are the search, book and cancel views.
- **Template:** HTML files that present the data. All pages extend a common base.html, which holds the navigation bar, the message area and the footer.

When a user opens a page, Django's URL router selects the matching view. The view reads or writes data through the models, passes the result to a template, and the rendered HTML is sent back to the browser.

4.2 Project Structure

The project follows the standard layout created by Django, with one app named rentals.

```
sarthigo/
  manage.py
  requirements.txt
  sarthigo/            (project settings and root URLs)
    settings.py
    urls.py
    wsgi.py
  rentals/             (the application)
    models.py          Car, Booking
    views.py           page logic
    forms.py           search, signup, driver details
    urls.py            routes
    admin.py           admin registration
    management/commands/seed_cars.py
    templates/rentals/
      base, home, login, register, car_list,
      confirm_booking, booking_success,
      my_bookings, admin_dashboard
```

The outer sarthigo package holds the configuration that is common to the whole site, while the rentals app contains everything specific to car rental. Keeping the logic inside an app makes it reusable and easy to find.

4.3 Database Design

The database has two project tables, Car and Booking, which are linked to Django's User table.

Table: Car

Field	Type	Description
id	Integer, primary key	Automatically generated identifier
name	Text (up to 100 characters)	Name of the car, for example Honda City
capacity	Positive integer	Number of seats
price_per_day	Decimal (8 digits, 2 decimals)	Rental price for one day
image_url	URL, optional	Link to a picture of the car

Table: Booking

Field	Type	Description
id	Integer, primary key	Booking number
user	Foreign key to User	The customer who made the booking
car	Foreign key to Car	The car that was booked
start_date	Date	First day of the rental
end_date	Date	Last day of the rental
total_cost	Decimal (10 digits, 2 decimals)	Price per day multiplied by number of days
is_approved	Boolean, default true	True for an active booking, false once canceled
driver_name	Text (up to 100 characters)	Full name of the driver
driver_dob	Date	Date of birth of the driver
driver_license	Text (up to 30 characters)	Driving licence number
created_at	Date and time	Set automatically when the booking is created

4.4 Relationships

One car can have many bookings, and each booking belongs to exactly one car, so Car and Booking are in a one-to-many relationship. In the same way, one user can make many bookings, and each booking belongs to one user. Both relationships are created with foreign keys, and when a user or car is deleted its bookings are deleted with it.

A booking is cancelled by setting is_approved to false. The row stays in the table, which keeps the history for the customer and the owner, and the availability search simply ignores bookings that are not approved.

4.5 URL Design

Every page of the site is reached through a named URL, which allows templates and views to refer to pages without hard-coding addresses.

URL path	View	Access	Purpose
/	home	Everyone	Hero section and search form
/cars/	car_list	Everyone	Cars available for the dates and seats
/book/id/	confirm_booking	Logged in	Driver details, terms and confirmation
/booking/id/done/	booking_success	Logged in, owner only	Confirmation page
/my-bookings/	my_bookings	Logged in	History of the user's bookings
/register/	register	Everyone	Create an account
/login/	Django LoginView	Everyone	Log in
/logout/	Django LogoutView	Logged in	Log out (sent as a POST request)
/admin-dashboard/	admin_dashboard	Staff only	All bookings from all users
/admin-dashboard/cancel/id/	cancel_booking	Staff only, POST	Cancel a booking
/admin/	Django admin	Superuser	Built-in data administration

4.6 Booking Workflow

A booking passes through five stages, which match the five screens that the customer sees.

Step	Stage	What happens
1	Search	The customer enters dates and seats on the home page.

Step	Stage	What happens
2	Results	Only cars that are free for those dates are listed with the total price.
3	Login	The customer logs in or registers; this is required before booking.
4	Confirm	The customer enters driver details and accepts the terms.
5	Done	A confirmation page is shown and the booking appears in My Bookings.

The total cost is price per day multiplied by the number of days, and both the start day and the end day count. For example, a Honda City at Rs. 2400 per day booked from the 10th to the 12th is three days, so the total is Rs. 7200.

The search dates are carried from page to page in the web address (the query string), so the confirmation page can recalculate the cost and verify availability again without storing anything in the session.

4.7 Security Design

Security was considered from the start and uses the protections that Django already provides.

- **Password storage:** passwords are never stored as plain text. Django hashes them, and the password validators reject short, common and fully numeric passwords.
- **CSRF protection:** every form that changes data contains a CSRF token, and the middleware rejects requests without a valid one.
- **Access control:** the login-required decorator protects booking pages, and a staff-only check protects the dashboard and the cancel action.
- **Safe HTTP methods:** cancelling a booking and logging out are done only with POST requests, so that they cannot be triggered by a simple link or an image tag.
- **Ownership check:** the confirmation page loads a booking only if it belongs to the logged-in user.
- **SQL injection:** all database access goes through the ORM, which uses parameterised queries.
- **Server-side validation:** all dates, seat counts and driver details are validated again on the server.

5. Implementation

This chapter explains the most important parts of the code. The complete source is supplied with the project.

5.1 Project Settings

The settings file registers the rentals app, selects SQLite as the database and defines where unauthenticated users are sent. The time zone is set to Asia/Kolkata so that the date used for the past-date check matches the local date.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rentals',
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

TIME_ZONE = 'Asia/Kolkata'
LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'home'
LOGOUT_REDIRECT_URL = 'home'
```

5.2 Models

The models describe the two tables designed in Chapter 4. The Booking class also defines two helper properties. The days property returns the inclusive number of days, and the status property returns the text Approved or Canceled for display in the tables.

```
class Car(models.Model):
    name = models.CharField(max_length=100)
    capacity = models.PositiveIntegerField(help_text='Number of seats')
    price_per_day = models.DecimalField(max_digits=8, decimal_places=2)
    image_url = models.URLField(blank=True)

    class Meta:
        ordering = ['price_per_day']


class Booking(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='bookings')
    car = models.ForeignKey(Car, on_delete=models.CASCADE, related_name='bookings')
    start_date = models.DateField()
    end_date = models.DateField()
    total_cost = models.DecimalField(max_digits=10, decimal_places=2)
    is_approved = models.BooleanField(default=True)
    driver_name = models.CharField(max_length=100)
    driver_dob = models.DateField()
    driver_license = models.CharField(max_length=30)
    created_at = models.DateTimeField(auto_now_add=True)

    @property
    def days(self):
        return (self.end_date - self.start_date).days + 1

    @property
    def status(self):
        return 'Approved' if self.is_approved else 'Canceled'
```

Cars are ordered by price so that the cheapest option appears first in the search results. Bookings are ordered with the newest first, which suits both My Bookings and the dashboard.

5.3 Checking Availability

The central rule of the application is the overlap test. Two date ranges overlap when each one starts on or before the day the other one ends. A car is therefore busy if it has an approved booking whose start date is on or before the requested end date and whose end date is on or after the requested start date.

```
def available_cars(start, end, seats):
    busy = Booking.objects.filter(
        is_approved=True,
        start_date__lte=end,
        end_date__gte=start,
    ).values_list('car_id', flat=True)
    return Car.objects.filter(capacity__gte=seats).exclude(id__in=busy)
```

The function returns every car that has at least the requested number of seats and is not in the busy list. Canceled bookings have is_approved set to false, so they are ignored and the car becomes available again immediately. The same function is called again on the confirmation page to prevent a double booking if two users choose the same car at nearly the same time.

5.4 Calculating the Cost

Both the start day and the end day are counted, so a one-day rental has the same start and end date and costs one day's price.

```
def calc_total(car, start, end):
    days = (end - start).days + 1
    return car.price_per_day * days, days
```

Because the price is a Decimal value, there are no floating-point rounding errors in the money calculation.

5.5 Forms and Validation

Three form classes handle all user input. The search form checks the date logic in its clean method, which runs after each individual field has been validated.

```
class SearchForm(forms.Form):
    start_date = forms.DateField(widget=forms.DateInput(attrs={'type': 'date'}))
    end_date = forms.DateField(widget=forms.DateInput(attrs={'type': 'date'}))
    seats = forms.IntegerField(min_value=1, max_value=12, initial=4)

    def clean(self):
        data = super().clean()
        start, end = data.get('start_date'), data.get('end_date')
        if start and end:
            if start < date.today():
                raise forms.ValidationError('Start date cannot be in the past.')
            if end < start:
                raise forms.ValidationError('End date cannot be before the start date.')
        return data
```

The sign-up form extends Django's UserCreationForm and adds a required e-mail field. The driver form collects the name, date of birth and licence number, requires the terms checkbox, and rejects drivers younger than 18 years.

```
def clean_driver_dob(self):
    dob = self.cleaned_data['driver_dob']
    today = date.today()
    age = today.year - dob.year - ((today.month, today.day) < (dob.month, dob.day))
    if age < 18:
        raise forms.ValidationError('Driver must be at least 18 years old.')
    return dob
```

The age expression subtracts one year when the birthday has not yet occurred in the current year, so the age is exact.

5.6 Views

The home view shows the search form. When a valid search arrives, it redirects to the car list with the dates and seats in the address. The car list view validates the data again on the server, so a user who edits the address by hand or bypasses the JavaScript still cannot search with invalid dates.

```
def car_list(request):
    form = SearchForm(request.GET)
    if not form.is_valid():
        for errors in form.errors.values():
            for e in errors:
                messages.error(request, e)
        return redirect('home')
    d = form.cleaned_data
    cars = available_cars(d['start_date'], d['end_date'], d['seats'])
    days = (d['end_date'] - d['start_date']).days + 1
    rows = [{'car': c, 'total': c.price_per_day * days} for c in cars]
    return render(request, 'rentals/car_list.html',
                  {'rows': rows, 'days': days, **d})
```

The booking view is the most important one. It requires login, validates the dates again, confirms that the car is still free, calculates the total, shows the driver form and, when the form is valid, creates the booking.

```
@login_required
def confirm_booking(request, car_id):
    car = get_object_or_404(Car, id=car_id)
    search = SearchForm(request.GET)
    if not search.is_valid():
        messages.error(request, 'Please choose your dates again.')
        return redirect('home')
    start = search.cleaned_data['start_date']
    end = search.cleaned_data['end_date']

    if not available_cars(start, end, 1).filter(id=car.id).exists():
        messages.error(request, 'Sorry, that car was just booked for those dates.')
        return redirect('home')

    total, days = calc_total(car, start, end)
    form = DriverForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        d = form.cleaned_data
        booking = Booking.objects.create(
            user=request.user, car=car, start_date=start, end_date=end,
            total_cost=total, driver_name=d['driver_name'],
            driver_dob=d['driver_dob'], driver_license=d['driver_license'],
        )
        return redirect('booking_success', booking_id=booking.id)
    return render(request, 'rentals/confirm_booking.html',
                  {'car': car, 'form': form, 'start': start, 'end': end,
                   'days': days, 'total': total})
```

The remaining customer views are short. The success page and My Bookings both use the logged-in user, so one customer can never see another customer's bookings. The register view creates the account, logs the user in immediately and shows a welcome message.

5.7 Admin Dashboard and Cancellation

Staff pages use a small decorator that allows only active users with the staff flag. A user who is not staff is sent to the login page.

```
def staff_only(view):
    return user_passes_test(lambda u: u.is_active and u.is_staff,
                            login_url='login')(view)


@staff_only
def admin_dashboard(request):
    bookings = Booking.objects.select_related('car', 'user')
    return render(request, 'rentals/admin_dashboard.html', {'bookings': bookings})


@staff_only
@require_POST
def cancel_booking(request, booking_id):
    booking = get_object_or_404(Booking, id=booking_id)
    booking.is_approved = False
    booking.save(update_fields=['is_approved'])
    messages.success(request, f'Booking #{booking.id} was canceled.')
    return redirect('admin_dashboard')
```

The select_related call fetches the car and user together with each booking in one query, which avoids running an extra query for every table row. The cancel view accepts only POST requests and changes only the is_approved field. The booking row is kept, and the car is free again for those dates.

5.8 Templates

All pages extend base.html, which contains the Tailwind CDN script, the navigation bar, the area where success and error messages are displayed, and the footer. The navigation bar changes according to the user. A visitor sees Login and Sign up, a customer sees My Bookings and a Logout button, and a staff member also sees the Admin Dashboard link.

The car list page shows each car as a card with its picture, seat count, price per day and the total for the chosen dates, followed by a Book now button. If no car is free, the page shows a friendly message and a link to try different dates.

The admin dashboard shows a table with the booking number, user, car, dates, total and a green Approved or red Canceled label. Each approved row has a Cancel button inside a small form that opens a confirm pop-up before it is submitted.

5.9 Browser-Side Date Check

The home page contains a short script. It sets the minimum date of both fields to today, keeps the end date from going earlier than the start date, and blocks the submit with an alert if the dates are invalid.

```
const today = new Date().toISOString().slice(0, 10);
start.min = today;
end.min = today;
start.addEventListener('change', () => { end.min = start.value || today; });

form.addEventListener('submit', (e) => {
  if (start.value < today) {
    alert('Start date cannot be in the past.');
    e.preventDefault();
  } else if (end.value < start.value) {
    alert('End date cannot be before the start date.');
    e.preventDefault();
  }
});
```

This gives the user instant feedback without a page reload. It is only a convenience, because the server repeats the same checks.

5.10 Sample Data Command

A custom management command named seed_cars adds eight sample cars so that the site can be demonstrated immediately. It uses get_or_create, so running it more than once does not create duplicates.

Car	Seats	Price per day (Rs.)
Maruti Alto	4	1200
Maruti Swift	5	1600
Hyundai i20	5	1800
Honda City	5	2400
Hyundai Creta	5	2800
Mahindra Scorpio	7	3200
Toyota Innova Crysta	7	3800
Tempo Traveller	12	6000

The owner can change prices, add new cars or replace the placeholder pictures later from the Django admin site.

6. User Interface Description

The interface uses a dark navigation bar and footer with an amber accent colour, white cards on a light grey background, and rounded corners and soft shadows. The layout adapts to narrow screens by stacking the columns.

6.1 Home Page

The home page has a hero section with the tagline Find Your Ride and a short sentence that explains the service. Beside it, a white card holds the search form with the fields Start date, End date and Seats needed, and a Search Cars button. If the browser-side check fails, an alert appears. If the server-side check fails, an error message is shown at the top of the page.

Figure 6.1: Home page with the search form (insert screenshot here).

6.2 Available Cars Page

This page repeats the chosen dates, the number of days and the seat filter as a heading. Below it, each free car is shown as a card with a picture, name, seats, price per day, the total price for the whole period and a Book now button. When no car matches, the page explains this and offers a link back to the search.

Figure 6.2: List of available cars (insert screenshot here).

6.3 Login and Sign-Up Pages

The login page asks for a username and password and shows a short error if they are wrong. A user who tried to book before logging in is returned to the booking page after login. The sign-up page asks for a username, e-mail and a password twice, and shows a clear message under each field that fails validation.

6.4 Confirm Booking Page

The page is split into two cards. The left card summarises the trip, with the car, start and end date, number of days, price per day and the total. The right card is the driver form with name, date of birth, licence number and the checkbox for the terms. Errors, such as a driver under 18 or an unchecked terms box, appear under the field that caused them.

Figure 6.3: Confirm booking page (insert screenshot here).

6.5 Booking Confirmation Page

After a successful booking, the user sees a green tick, the booking number, the car, the dates, the driver name and the total cost, with a button that leads to My Bookings.

6.6 My Bookings Page

My Bookings shows a table of the user's bookings with number, car, dates, total and a coloured status label. If the user has no bookings yet, a message with a link to find a car is displayed.

6.7 Admin Dashboard

The dashboard is available only to staff. It lists every booking from every user, along with the username, and a Cancel button on each approved booking. The button asks for confirmation before the booking is canceled, and the status changes to Canceled while the row stays in the table.

Figure 6.4: Admin dashboard showing all bookings (insert screenshot here).

7. Testing

7.1 Testing Strategy

The application was tested at three levels. First, individual rules such as date validation, availability and cost calculation were checked with chosen examples. Second, the complete customer journey was run from search to confirmation, and the staff journey from login to cancellation. Third, access control was tested by trying to open protected pages as a visitor and as a normal customer.

The journeys were also run automatically with Django's test client, which sends requests to the application exactly as a browser would and lets the test examine the responses and the database. After the tests, the temporary data was removed.

7.2 Test Cases

No.	Test case	Input / action	Expected result	Result
1	Open home page	Visit /	Page loads with the search form	Pass
2	Valid search	Future dates, 5 seats	List of cars with 5 or more seats	Pass
3	Start date in the past	Yesterday as the start date	Alert in browser; error from server	Pass
4	End before start	End date earlier than start	Search blocked with an error message	Pass
5	Book without login	Open the book page as a visitor	Redirect to login	Pass
6	Register	Valid username, e-mail, password	Account created, user logged in	Pass
7	Book a car	Valid driver details, terms checked	Booking saved and confirmation shown	Pass
8	Cost calculation	Honda City, 3 days at Rs. 2400	Total Rs. 7200	Pass
9	Terms not accepted	Terms box left empty	Error under the checkbox, no booking	Pass
10	Driver under 18	Date of birth less than 18 years ago	Error message, no booking	Pass
11	Booked car hidden	Search the same dates again	Booked car is not in the list	Pass
12	My Bookings	Open as the customer	Booking is shown as Approved	Pass
13	Dashboard as customer	Open the admin dashboard	Redirect to login	Pass
14	Dashboard as staff	Open as a superuser	All bookings are listed	Pass
15	Cancel booking	Staff presses Cancel and confirms	Status becomes Canceled, record kept	Pass
16	Car free again	Search the same dates after cancelling	The car appears in the list again	Pass

7.3 Result of Testing

All sixteen test cases gave the expected result. The tests confirmed that the availability rule works in both directions: a car disappears from the results after it is booked, and it returns after the booking is canceled. They also confirmed that protected pages cannot be opened without the correct role.

8. Installation and Execution

The commands below are run in a terminal, in the folder that contains manage.py.

Step	Command	Purpose
1	pip install -r requirements.txt	Install Django
2	python manage.py makemigrations rentals	Prepare the table definitions
3	python manage.py migrate	Create the tables in db.sqlite3
4	python manage.py seed_cars	Add the sample cars
5	python manage.py createsuperuser	Create the staff account
6	python manage.py runserver	Start the development server
7	Open http://127.0.0.1:8000/	Use the site in a browser

The terminal must stay open while the site is being shown, and the server is stopped with Ctrl + C. The staff dashboard is at the path /admin-dashboard/, and it is available after logging in with the superuser account. The built-in Django admin is at /admin/.

9. Limitations

- There is no online payment, so the total is shown but not collected on the site.
- Bookings are approved automatically. There is no step in which staff accept or reject a new request.
- A customer cannot cancel a booking personally; only staff can cancel.
- The system has no e-mail or SMS notifications.
- The licence number is stored as text and is not verified with any authority.
- Car pictures are links to external images, not files uploaded to the site.
- The site is intended for a single rental business in one city, with one pick-up location.
- Tailwind CSS is loaded from a CDN, so an internet connection is needed for the full styling.
- SQLite is suitable for a demonstration but not for many users writing at the same time.

10. Future Scope

- **Online payment:** integrate a payment gateway so that the booking is confirmed after payment, with refunds on cancellation.
- **Customer cancellation:** let customers cancel their own bookings before a cut-off time.
- **Notifications:** send a confirmation and reminders by e-mail or SMS.
- **Image upload:** let staff upload car photos directly and show several photos per car.
- **Search filters:** add filters for fuel type, transmission, price range and car category.
- **Ratings and reviews:** allow customers to rate a car and a trip after it is completed.
- **Reports for staff:** add charts for revenue, most booked cars and busy periods.

- **Locations:** support several pick-up points and driver-included bookings.
- **Production database and hosting:** move to PostgreSQL, serve static files properly, set DEBUG to false and deploy on a cloud server with HTTPS.
- **REST API and mobile app:** expose the data through an API so that a mobile app can be built on the same backend.

11. Conclusion

SarthiGo achieves the aim of the project: it lets a customer search for cars that are actually free on the chosen dates, book one with driver details, and keep a history of bookings, while staff can monitor and cancel bookings from one dashboard. The application is built with Python and Django using the Model-View-Template pattern, styled with Tailwind CSS and backed by SQLite.

The project gave practical experience in several important topics: designing models and relationships, writing the date-overlap rule for availability, validating input on both the browser and the server, using Django's authentication and permission system, and protecting forms with CSRF tokens. Keeping cancelled bookings with a flag instead of deleting them was a small design decision that makes the history reliable.

The project also showed that a small, well-structured application can solve a real everyday problem. With the improvements listed in Chapter 10, especially payment and notifications, SarthiGo can grow from a mini project into a service that a local car rental business could use.

12. References

1. Django Software Foundation, Django documentation, https://docs.djangoproject.com
2. Python Software Foundation, Python 3 documentation, https://docs.python.org/3/
3. Tailwind Labs, Tailwind CSS documentation, https://tailwindcss.com/docs
4. SQLite Consortium, SQLite documentation, https://www.sqlite.org/docs.html
5. Mozilla Developer Network, JavaScript and HTML reference, https://developer.mozilla.org
6. Course material, Python and Web Programming (202045612)